

Session 16 : Character Controller — FK et IK

Objectifs de la session

- Comprendre pourquoi un personnage ne se simule **pas** comme un rigid body — et ce qu'est un **character controller**.
- Construire le contrôleur canonique : **capsule cinématique**, **raycast au sol**, glissement le long des obstacles.
- Passer de la **cinématique directe** (FK) à la **cinématique inverse** (IK) : donner la position de la main et retrouver les angles des articulations.
- **Implémenter** deux algorithmes d'IK : **CCD** (Cyclic Coordinate Descent) et **FABRIK** (Forward And Backward Reaching Inverse Kinematics), avec leurs compromis.
- Voir l'IK appliquée au personnage : **foot placement**, **look-at**, **hand grab**, **active ragdoll**.
- **Faire le pont** vers le TP Véhicules (S17) : le raycast vehicle est un **character controller à roues**.

💡 Le passage au personnage

Depuis la Session 14, on simule des **objets** : boîtes qui tombent, boulets qui percutent, catapultes qui s'articulent. Le moteur décide de tout — gravité, collisions, impulsions. Un **personnage**, c'est l'inverse : c'est le **code** qui décide (avancer, sauter, tourner), et le moteur qui **vérifie** (y a-t-il un mur ? un sol ? une pente ?). C'est le **character controller** — l'outil le plus important du gameplay. Et une fois qu'on contrôle un corps, il reste à contrôler ses **articulations** : c'est là que la cinématique inverse (IK) entre en jeu.

Partie 1 : Pourquoi pas un rigid body ?

Le piège du rigid body

L'intuition du débutant : « un personnage, c'est un corps physique — je lui mets une capsule de rigid body et ça marche ». Résultat :

- Il **trébuche** — le moindre contact le fait basculer (il a une masse et une inertie).
- Il **glisse** sur les pentes (la friction décide, pas le joueur).
- Il **rebondit** contre les murs (restitution) au lieu de s'arrêter net.
- Il **dérape** au moindre virage — impossible à diriger avec précision.

Un rigid body est **simulé** : le moteur décide. Un personnage doit être **contrôlé** : le code décide, le moteur vérifie.

Character controller

Un **character controller** est un corps **cinématique** piloté par le code. À chaque frame :

1. Le code calcule la **vitesse voulue** (input du joueur).
2. Le contrôleur **déplace** la capsule selon cette vitesse.
3. Le moteur **vérifie** les collisions sur la trajectoire :
 - **Mur** ? On **glisse le long** du mur (on garde la composante parallèle).
 - **Sol** ? On **reste dessus** — et on peut **sauter**.
 - **Pente trop raide** ? On ne **monte pas**.
 - **Marche basse** ? On **monte dessus** automatiquement (step offset).

Le personnage ne reçoit **jamais** d'impulsion, ne **tourne** jamais au contact, ne **tombe** jamais de fatigue. Il fait **exactement** ce que le code lui dit — dans les limites de la géométrie.

Partout dans l'industrie.

- **Quake / Source** : le contrôleur historique — capsule, glissement sur les murs, marches gravies automatiquement.
- **Unity** : composant `CharacterController` (`Move()`).
- **Godot** : nœud `CharacterBody3D` (`move_and_slide()`).
- **Unreal** : `ACharacter` + `CharacterMovementComponent`.
- Aucun n'utilise un rigid body pour le déplacement — tous sont **cinématiques**.

Partie 2 : Anatomie du contrôleur

La capsule — pas la boîte

La forme standard est la **capsule** (deux sphères + un cylindre) :

- **Pas de coins** — une boîte s'accroche sur les seuils de portes et les arêtes de marches ; la capsule **glisse** naturellement.
- **Section ronde** — elle se faufile dans les couloirs étroits sans se coincer.
- **Extrémités arrondies** — elle monte les petites marches sans « sauter » visuellement.

Le rayon et la hauteur de la capsule déterminent la **silhouette de collision** du personnage — souvent plus généreuse que son modèle visuel.

La boucle de contrôle

```
# Godot — CharacterBody3D : la boucle canonique
var velocity := Vector3.ZERO
```

```
func _physics_process(delta):
    # 1. Input → vitesse voulue (le CODE décide)
    var input_dir := Input.get_vector("left", "right", "forward", "back")
    velocity.x = input_dir.x * SPEED
    velocity.z = input_dir.y * SPEED

    # 2. Gravité et saut — conditionnés au sol
    if not is_on_floor():
        velocity.y -= gravity * delta
    elif Input.is_action_just_pressed("jump"):
        velocity.y = JUMP_SPEED

    # 3. Déplacement + vérification (le MOTEUR vérifie)
    move_and_slide() # glisse le long des murs, détecte sol/pentes/marches
```

`move_and_slide()` fait tout le travail pénible : elle **déplace** la capsule, **détecte** les collisions, **projette** la vitesse restante le long des surfaces (glissement), et expose `is_on_floor()`, `is_on_wall()`, `get_slide_collision()`.

💡 Grounded — le raycast au sol

La question « suis-je au sol ? » est le **cœur** du contrôleur : elle conditionne le saut, la friction, l'animation. La réponse universelle : un **raycast** vers le bas, un peu plus long que la hauteur du pied :

```
// Rapier — suis-je au sol ?
const ray = new RAPIER.Ray(feetPosition, { x: 0, y: -1, z: 0 });
```

```
const hit = world.castRay(ray, 0.15, true); // 15 cm de tolérance
const grounded = hit !== null;
```

C'est exactement le raycast de la Session 15 — le même outil qui, la semaine prochaine, fera **rouler** votre véhicule.

💡 Et les contraintes dans tout ça ?

Un squelette, c'est une **chaîne de contraintes** — l'épaule, le coude, le poignet sont des revolute joints (S15). Jusqu'ici on a piloté ces chaînes **en FK** : on donne les angles, l'extrémité suit. On va maintenant les piloter **en IK** : on donne le point à atteindre, les angles suivent.

Partie 3 : Forward vs Inverse Kinematics

Forward Kinematics (FK)

En FK, on **connaît** les angles des articulations et on **calcule** la position de l'extrémité. C'est le **sens naturel** d'une chaîne articulée : on part de la base, on applique chaque rotation, et l'extrémité **suit**.

$$\vec{p} = f(\theta_1, \theta_2, \theta_3, \dots)$$

```
// FK – chaîne à 2 segments dans le plan
function forwardKinematics(angles, lengths) {
  let x = 0, y = 0, totalAngle = 0;
  const points = [{ x: 0, y: 0 }];
  for (let i = 0; i < angles.length; i++) {
    totalAngle += angles[i];
    x += lengths[i] * Math.cos(totalAngle);
    y += lengths[i] * Math.sin(totalAngle);
    points.push({ x, y });
  }
  return points; // positions de chaque articulation
}
```

La FK est **directe** : une seule solution, calculable en $O(n)$ pour n articulations. C'est ce qu'on fait depuis le début du cours — un bras de catapulte, un pendule, une roue.

Inverse Kinematics (IK)

En IK, on **connaît** la position cible de l'extrémité (la main, le pied) et on **cherche** les angles des articulations qui amènent l'extrémité à cette cible. C'est le **problème inverse** :

$$\theta_1, \theta_2, \theta_3, \dots = f^{-1}(\vec{p}_{\text{cible}})$$

Difficultés :

- **Plusieurs solutions** : un bras à 3 articulations peut atteindre la même cible avec des poses très différentes (coude en avant, coude en arrière, etc.).
- **Aucune solution** : si la cible est **hors de portée** (plus loin que la somme des longueurs), aucune pose ne fonctionne.
- **Pas de solution analytique simple** : pour $n \geq 4$ articulations, on ne peut pas **inverser** f directement — on utilise des **algorithmes itératifs**.

💡 Quand utilise-t-on l'IK ?

L'IK est partout où le **but** est plus important que le **moyen** :

- **Foot placement** : le pied doit toucher le sol à un point précis — l'IK calcule la pose de la jambe.
- **Hand grab** : la main doit atteindre un objet — l'IK calcule le bras.
- **Active ragdoll** : maintenir une pose (debout, assis) tout en subissant la physique.
- **Look-at** : la tête doit regarder une cible — l'IK calcule l'orientation du cou.

Partie 4 : CCD — Cyclic Coordinate Descent

CCD — principe

Le CCD **ajuste les articulations une par une**, de l'extrémité vers la base. À chaque étape, on **tourne** une articulation pour **aligner** l'extrémité avec la cible, **vue depuis cette articulation**.

```
// CCD — une itération complète (de l'extrémité vers la base)
function ccdSolve(joints, target, iterations) {
    const n = joints.length;
    for (let iter = 0; iter < iterations; iter++) {
        // De l'avant-dernière articulation jusqu'à la base
        for (let i = n - 2; i >= 0; i--) {
            // Vecteur : articulation → extrémité
            const toEnd = sub(joints[n - 1], joints[i]);
            // Vecteur : articulation → cible
            const toGoal = sub(target, joints[i]);

            // Angle entre les deux vecteurs
            const angle = angleBetween(toEnd, toGoal);
            // Tourner toutes les articulations *après* i
            for (let j = i + 1; j < n; j++) {
                joints[j] = rotateAround(joints[j], joints[i], angle);
            }
        }
    }
}
```

Avantage : simple à implémenter, converge vite (souvent 5-20 itérations), gère naturellement les **limites d'angle** (on clamp la rotation à chaque étape).

Inconvénient : les mouvements sont **peu naturels** — l'extrémité se « tortille » car chaque articulation est ajustée **indépendamment**. Le résultat **fonctionne** mais ne ressemble pas à un vrai bras.

💡 CCD et les limites d'angle

Le CCD est **très** utilisé en pratique **grâce** à sa gestion des limites. Comme on ajuste chaque articulation **indépendamment**, on peut **clamp** la rotation de chaque joint à son amplitude maximale **immédiatement**. Les algorithmes **globaux** (FABRIK) gèrent les limites **après coup**, ce qui est plus complexe. C'est pourquoi le CCD reste populaire pour les **bras robotiques** (où chaque axe a une butée mécanique).

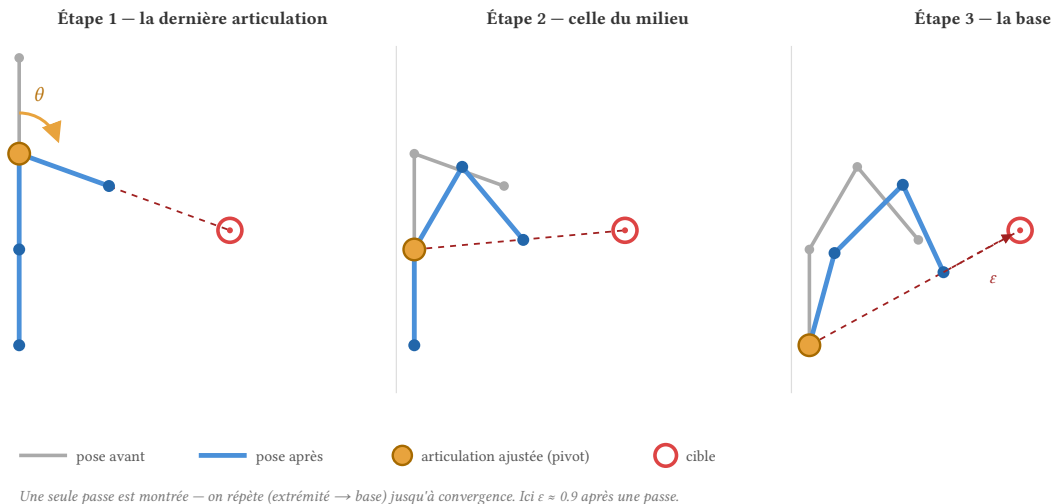


Figure 1: CCD : on ajuste les articulations **une par une**, de l'extrémité vers la base. À chaque étape, on fait **pivoter** toute la chaîne en aval pour aligner l'extrémité avec la cible, vue depuis l'articulation ajustée. Une seule passe est montrée — on répète jusqu'à convergence.

Partie 5 : FABRIK — Forward And Backward Reaching IK

FABRIK — principe

Le FABRIK ne manipule **pas** les angles — il manipule directement les **positions** des articulations. Deux passes par itération :

1. **Forward** : on **rapproche** l'extrémité de la cible, puis on **corrige** chaque articulation pour respecter les longueurs de segments — **en remontant** vers la base.
2. **Backward** : on **rapproche** la base de sa position d'origine (sinon le bras « flotte »), puis on **corrige** les longueurs **en descendant** vers l'extrémité.

```
// FABRIK — une itération
function fabrikSolve(joints, target, origin, lengths) {
  const n = joints.length;
  const distToTarget = distance(joints[0], target);
  const totalLength = lengths.reduce((a, b) => a + b, 0);

  // Cible hors de portée : étirer vers la cible
  if (distToTarget > totalLength) {
    for (let i = 0; i < n - 1; i++) {
      const r = distance(joints[i], target);
      const lambda = lengths[i] / r;
      joints[i + 1] = lerp(joints[i], target, lambda);
    }
    return;
  }

  // --- Phase FORWARD : cible → base ---
  joints[n - 1] = target; // extrémité sur la cible
  for (let i = n - 2; i >= 0; i--) {
    const r = distance(joints[i], joints[i + 1]);
    const lambda = lengths[i] / r;
    // Rapprocher joints[i] pour respecter la longueur
    joints[i] = lerp(joints[i + 1], joints[i], lambda);
  }
}
```

```

}

// --- Phase BACKWARD : base → extrémité ---
joints[0] = origin; // base remise à sa position
for (let i = 0; i < n - 1; i++) {
  const r = distance(joints[i], joints[i + 1]);
  const lambda = lengths[i] / r;
  joints[i + 1] = lerp(joints[i], joints[i + 1], lambda);
}
}

```

Avantage : mouvements **naturels** (toutes les articulations bougent **ensemble**), très rapide (souvent 10 itérations suffisent), pas de calcul d'angles.

Inconvénient : ne gère **pas** les limites d'angle directement — il faut un post-traitement pour **clamp** les orientations après chaque itération.

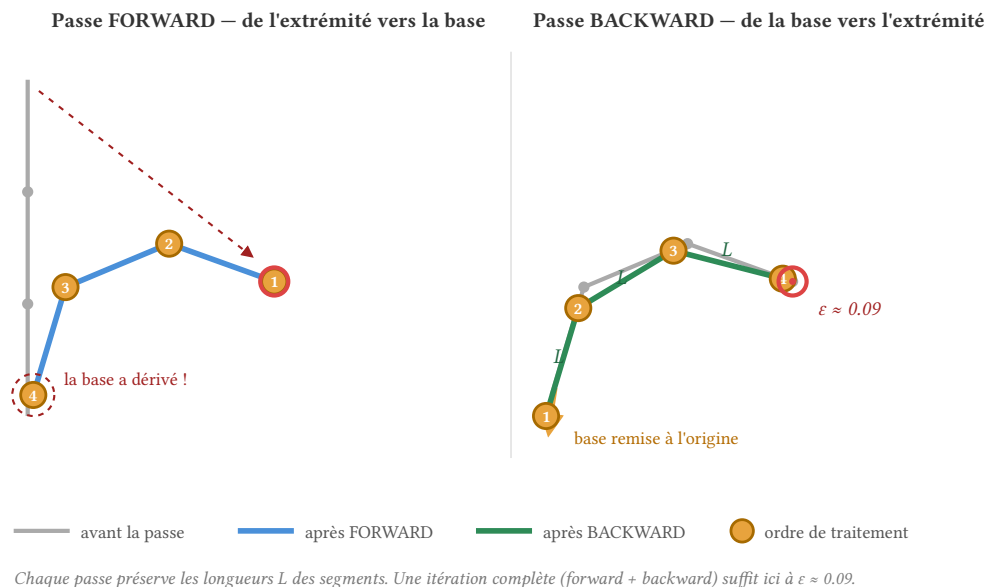


Figure 2: FABRIK : la passe **FORWARD** place l'extrémité sur la cible puis repositionne les articulations en remontant vers la base — qui **dérive**. La passe **BACKWARD** ramène la base à l'origine puis repositionne en descendant vers l'extrémité. Les longueurs L des segments sont **toujours** préservées — une itération complète suffit ici à $\varepsilon \approx 0.09$.

⚠ FABRIK et les limites d'angle — le post-traitement

Le FABRIK pur ignore les limites articulaires : un coude peut se tordre à 360° . Pour respecter des limites, on **projette** chaque orientation de segment dans le **cône autorisé** après chaque itération. C'est plus complexe que le CCD (où on clamp directement), mais le résultat est plus **naturel**. En pratique, beaucoup de jeux utilisent FABRIK **sans** limites pour les **pieds** (où les limites sont peu contraignantes) et CCD **avec** limites pour les **bras** (où le coude a une amplitude limitée).

Partie 6 : Applications de l'IK

Cas d'usage concrets.

- **Foot placement (marche sur terrain)** : on raycast vers le bas depuis la hanche, on obtient le point de contact, et l'IK place le pied **sur** ce point — la jambe s'adapte à la pente, aux marches, aux cailloux.
- **Active ragdoll** : un personnage mort qui **essaie** de rester debout. L'IK maintient une pose cible (debout) pendant que la physique le pousse — le résultat est un corps qui **lutte** contre les chutes, comme un zombie.
- **Hand grab** : le joueur clique sur un objet, l'IK calcule la pose du bras pour que la main atteigne le point de préhension. Sans IK, il faudrait **animer** chaque prise à la main.
- **Look-at (tête/canon)** : la tête ou le canon d'une tourelle **poursuit** une cible. L'IK calcule l'orientation du cou ou de l'axe pour pointer vers la cible.
- **Procédural animation** : remplacer les animations **précalculées** par de l'IK en temps réel — le personnage **réagit** au monde au lieu de rejouer un clip.

Partie 7 : Le pont vers les véhicules

💡 Le véhicule est un character controller à roues

Le TP de la Session 17 — le **Raycast Vehicle** — applique **exactement la philosophie** d'aujourd'hui : le code décide (throttle, direction), les raycasts vérifient (où est le sol sous chaque roue ?). Au lieu d'une capsule, un **châssis** ; au lieu d'un raycast, **quatre** — un par roue, chacun avec sa suspension :

- La **suspension** est un ressort vertical : on veut que la roue reste à une distance L du châssis, mais on **autorise** la compression.
- Le **grip** empêche la roue de **glisser** latéralement — sans quoi la voiture dérape à chaque virage.
- Le **raycast** est la **détection** : c'est le raycast **grounded** d'aujourd'hui, appliqué 4 fois par frame.

Aperçu du modèle Raycast Vehicle

Au lieu de simuler 4 roues comme des **rigid bodies** (complexe, instable), on traite le châssis comme un **seul** body et les roues comme des **raycasts** :

```
// Aperçu – suspension d'une roue (TP S17)
for (const wheel of wheels) {
    // 1. Raycast depuis la roue vers le bas
    const ray = new RAPIER.Ray(wheelPos, { x: 0, y: -1, z: 0 });
    const hit = world.castRay(ray, suspensionRest, true);

    if (hit) {
        // 2. Compression : 0 (étendu) → 1 (comprimé)
        const compression = 1 - hit.timeOfImpact / suspensionRest;
        // 3. Force de suspension (ressort + amortisseur)
        const springForce = suspensionK * compression;
        const damperForce = suspensionDamping * wheelVelocityY;
        const force = springForce - damperForce;
        // 4. Appliquer la force au châssis au point de contact
        chassisBody.addForceAtPoint({ x: 0, y: force, z: 0 }, wheelPos);
    }
}
```

La **suspension** est un **ressort** (Session 7) **résolu comme une contrainte** (aujourd'hui) **détecté par raycast** (Bientôt).

Partie 8 : Démonstration — le bras IK

💡 L'exemple du cours

exemples/session16_ik.html + session16_ik.js — un bras à 4 segments, une cible : **l'anneau suit la souris**, le bras le poursuit. La GUI bascule entre **CCD** et **FABRIK** — comparez le nombre d'itérations et la forme du bras sur la même trajectoire. L'anneau devient **rouge** quand la cible est hors de portée : le bras s'étire alors vers elle au lieu de l'atteindre.

Partie 9 : Démo Godot — l'araignée procédurale

💡 Le script

scenes/spider_demo.tscn + scripts/spider_demo.gd (projet demo-physics) — une araignée à 8 pattes pilotée au ZQSD/WASD. Le corps est un CharacterBody3D ; chaque patte est une chaîne **hanche** → **genou** → **pied** résolue en FABRIK. Les pieds **restent plantés au sol** pendant que le corps avance ; quand une patte s'étire trop, le pied se décolle et se re-planté devant. C'est la synthèse complète de la session : character controller + raycast + IK.

Idée 1 — le corps est un character controller

Rien de nouveau : le corps est exactement la boucle de la Partie 2 — input → vitesse voulue → move_and_slide(). Deux détails en plus :

- Le corps **tourne doucement** vers sa direction de marche (lerp_angle sur le yaw). Comme les ancres des pattes sont en **espace local** du corps, les pattes « suivent » la rotation gratuitement.
- La capsule de collision est **couchée** (rotation de 90°) plutôt qu'aplatie par scale — Jolt n'accepte pas l'échelle non-uniforme sur les formes de collision.

Idée 2 — le pied est un point du monde, pas du corps

Le truc central du foot placement : chaque pied mémorise sa position **en coordonnées monde**. Quand le corps avance, le pied ne bouge **pas** — il reste cloué au sol. À chaque frame on compare le pied à sa **position de repos**, calculée par un raycast vers le bas depuis un point devant la hanche :

```
# Position de repos de la patte i : le sol sous un point
# à REST_DIST de la hanche, dans sa direction d'éventail
var hip_w: Vector3 = _body.global_transform * _hip_local[i]
var dir_w: Vector3 = (_body.global_transform.basis * _rest_local[i]).normalized()
var from := hip_w + dir_w * REST_DIST + Vector3.UP * 1.5
var q := PhysicsRayQueryParameters3D.create(from, from + Vector3.DOWN * 4.0)
var hit := _space.intersect_ray(q)
```

C'est le **même raycast** que le grounded de la Partie 2 — sauf qu'on l'utilise pour trouver **où poser le pied**, pas pour savoir si on est au sol. Bonus : comme le raycast trouve le **vrai sol** (y compris les boîtes du décor), l'araignée grimpe sur les obstacles sans code supplémentaire.

Idée 3 — le solveur est le FABRIK du cours

_solve_leg() est fabrikSolve() transposé en 3D : chaîne à 3 joints (hanche = base mobile, pied = cible fixe). Passe **forward** — le pied est pinné sur sa cible, on remonte vers la hanche en

corrigeant les longueurs. Passe **backward** — la hanche est re-pinnée sur le corps, on redescend vers le pied. Cas hors de portée : la patte s'étire vers le pied, comme dans l'exercice.

```
for _it in FABRIK_ITER:
    # FORWARD : cible → base
    pts[2] = foot
    for k in range(1, -1, -1):
        var r := pts[k].distance_to(pts[k + 1])
        pts[k] = pts[k + 1].lerp(pts[k], lengths[k] / r)
    # BACKWARD : base → extrémité
    pts[0] = hip
    for k in range(0, 2):
        var r := pts[k].distance_to(pts[k + 1])
        pts[k + 1] = pts[k].lerp(pts[k + 1], lengths[k] / r)
```

La subtilité 3D : en 2D, le genou n'a que deux positions possibles ; en 3D, il a tout un **cercle** de positions valides autour de l'axe hanche→pied. Sans contrainte, il « flippe » d'une frame à l'autre. Solution standard — le **pole vector** : après la résolution, on tourne le genou sur ce cercle pour qu'il pointe vers l'extérieur et le haut. Tourner sur le cercle préserve les deux longueurs, donc le résultat reste exact.

Idée 4 — la démarche n'est PAS de l'IK

Point pédagogique important : l'IK répond à « **où** va le genou, étant donné la hanche et le pied ? ». Elle ne dit rien sur **quand** lever le pied. C'est une **machine à états** séparée — le planificateur :

1. **Pied planté** : si $\text{distance}(\text{pied}, \text{repos}) > \text{STEP_THRESHOLD}$ → déclencher un pas.
2. **Pas en cours** : interpoler le pied de l'ancienne vers la nouvelle position, avec un arc $\sin(t \cdot \pi) \cdot \text{STEP_HEIGHT}$ pour qu'il se **soulève**.
3. **Atterrissage** : le pied redevient un point monde fixe.

Deux raffinements pour le réalisme :

- **Cible prédictive** : on vise $\text{repos} + \text{vitesse} \times \text{durée du pas}$ — le pied atterrit là où il **sera** utile, pas là où il aurait fallu le mettre.
- **Démarche alternée** : les pattes sont en deux groupes diagonaux ; un seul groupe peut lever à la fois, et la main passe quand tous ses pieds sont au sol — l'effet « vague » d'une vraie araignée, au lieu de tout lever en bloc.

Ce que la démo illustre.

- **Séparation planificateur / solveur** : la machine à états décide **quand** bouger, l'IK calcule **comment** — c'est l'architecture des vrais systèmes d'animation procédurale.
- **Le pied comme ancre monde** : la pose « verrouillée au sol » n'est pas une contrainte physique — c'est juste une position monde qu'on refuse de bouger.
- **Le raycast partout** : grounded (Partie 2), foot target (ici), suspension des roues (S17) — le même outil de **requête**, trois rôles.
- **Le pole vector** : la solution élégante au problème « l'IK en 3D a trop de solutions » — on choisit la solution **plausible anatomiquement**.

Exercice 1 — CCD sur le bras

Mission

Ouvrez `examples/session16_ik.html`. L'anneau suit la souris ; le bras le poursuit avec l'algorithme sélectionné dans la GUI. Pour cet exercice, on se concentre sur le **CCD**.

1. Dans `session16_ik.js`, trouvez la fonction `ccdSolve()`. Elle est **incomplète** — la rotation des articulations est manquante.
2. Implémentez la **rotation** : pour chaque articulation i (de l'extrémité vers la base), calculez l'angle entre (articulation $\rightarrow$ extrémité) et (articulation $\rightarrow$ cible), puis **tournez** toutes les articulations **après** i de cet angle. Attention — l'extrémité **bouge** après chaque rotation : relisez `joints[n-1]` à chaque étape.
3. Testez avec la cible à différentes positions. Combien d'itérations suffisent pour atteindre la cible à 0.01 près ?

Exercice 2 — FABRIK et la cible hors de portée

Mission

Toujours dans `session16_ik.js`, trouvez `fabrikSolve()`.

1. Le cas **hors de portée** est déjà géré (le bras s'étire vers la cible — l'anneau devient rouge). Vérifiez-le en plaçant la cible **au-delà** de la longueur totale.
2. Implémentez le cas **atteignable** : les deux passes (forward + backward). Attention — la phase **backward** doit **replacer la base** à son origine, sinon le bras **flotte**.
3. Basculez entre CCD et FABRIK dans la GUI : FABRIK converge-t-il plus vite que CCD sur la même cible ? (Comparez le compteur d'itérations.)

Synthèse

Ce qu'il faut retenir

- **Un personnage n'est pas un rigid body** : il trébuche, glisse, rebondit. On le contrôle avec un **character controller** — corps **cinématique** piloté par le code, collisions **vérifiées** par le moteur.
- **La boucle du contrôleur** : `input` $\rightarrow$ vitesse voulue $\rightarrow$ déplacement $\rightarrow$ **glissement** le long des obstacles. La question « suis-je au sol ? » se règle avec un **raycast vers le bas** — le même qui fera rouler le véhicule en S17.
- **FK** : angles $\rightarrow$ positions (direct, une solution). **IK** : cible $\rightarrow$ angles (inverse, plusieurs solutions, itératif).
- **CCD** : ajuste les articulations **une par une** (de l'extrémité vers la base). Simple, gère les limites, mouvements peu naturels.
- **FABRIK** : ajuste les **positions** en deux passes (forward/backward). Rapide, mouvements naturels, limites en post-traitement.
- **Applications** : foot placement, look-at, hand grab, active ragdoll — l'IK adapte le squelette au monde.

💡 La suite

Session 17 : le **TP Véhicules**. On assemble un châssis, 4 raycasts, une suspension, un moteur et une direction — **drivable**. C'est le **projet final** du cours, et il **réutilise tout** : rigid bodies (S14), joints et raycasts (S15), character controller et IK (aujourd'hui). Chargez vos batteries — à la prochaine session, on roule en numérique.